

CSV, Comma Separated Values, is a plain text file format used to store structured tabular data. In a CSV file, a line represents a row in a table. Each line consists of a comma-separated value, representing a column in a row. A CSV file is saved with a .csv extension.

In this tutorial, we'll cover how to read, write, and manipulate CSV files using Python.

What are CSV Files Used For?

CSV files are extensively used across various applications and platforms. Some of the advantages of CSV files are listed below:

Data Analysis: CSV files are used to analyze data and to manipulate datasets for analysis.

Spreadsheet Applications: CSV files allow easy data manipulation and sharing using various spreadsheet applications, such as Microsoft Excel and Google Sheets.

Customer Relationship Management (CRM): CSV files are used to manage and analyze data, such as managing contact lists, campaign data, and customer management.

E-commerce and Inventory Management: CSV files help manage product listings and update inventory to allow data to be coordinated in bulk.

Imagine you are a Data Analyst at a startup. You regularly receive data from many sources. To manage the data, you start by storing it in a CSV file and processing it using a programming language such as Python.

Getting Started

Before you start working with CSV files in Python, it's crucial to set up your environment properly.

If you don't have Python installed on your system, visit [this link](#) to install it. To work with CSV files in Python, we will use the CSV module part of the standard library (no need to install it). We import the CSV module like so:

```
import csv
```

Now we have the environment ready to use the csv module. Let's learn what operations we can perform.

Overview of basic File Handling Operations supported by Python for CSV Files

We can perform operations like adding a new column or filtering out the data using a Python program.

First, let's learn the basic operations like opening, reading, and writing the CSV files.

How to Open CSV Files?

Let's see how we can open a CSV file using Python. A CSV file can be opened using the built-in function `open()` with the appropriate mode like ('r' for reading, 'w' for writing, or 'a' for appending). It's better to use the `with` statement that automatically handles the closing of the file even if any error occurs.

Let's look at an example to understand how we can open CSV files with Python. In a CSV file named `example.csv`, we have the following data:

```
Name,Age,Department
Alice,30,HR
Bob,24,IT
Charlie,28,Finance
```

The following code opens the `example.csv` file using the `open()` function and with read mode.

Then we use the `next()` function to skip the header row and a `for` loop to iterate over each row in the CSV file:

```
import csv

# Open the CSV file in read mode
with open('example.csv', mode='r') as file:
    # Create a CSV reader object
    csv_reader = csv.reader(file)

    # Skip the header row (if there is one)
    next(csv_reader, None)

    # Iterate over each row in the CSV file
    for row in csv_reader:
        print(row)
```

How to Read CSV Files?

To see how we can read CSV files with Python, let's look at an example. In a CSV file named `example.csv`, we have the following information:

```
Name,Age,Department
Alice,30,HR
Bob,24,IT
Charlie,28,Finance
```

We write a Python script to read this CSV file and print its contents like so:

```
import csv
```

```
# Open the CSV file
with open('example.csv', mode= 'r') as file:
    # Create a CSV reader object
    csv_reader = csv.reader(file)

    # Read the header
    header = next(csv_reader)
    print(f"Header: {header}")

    # Read each row of the CSV file
    for row in csv_reader:
        print(f"Row: {row}")
```

In this code, we create a **CSV reader object using csv.reader(file)**, which reads a CSV file and returns each row as a list of strings.

Next, we read the header of the CSV file using the **next(csv_reader) function** which is used to retrieve the next item from an iterator. A header in a CSV file is the first row that contains the names of the columns, providing a label for each column's data.

Then, we loop through the remaining rows in the CSV file, printing each row as a list of strings. This approach allows us to easily read and process CSV data in Python.

How to Write to a CSV File in Python?

To write the data into a CSV file, we first open the file in write mode using the with statement and then create a writer object using the csv.writer(file) to allow us to write the data into the file.

To understand it more concisely, let's look at an example. In a CSV file named output.csv, we have the following information:

```
Name,Age,Department
Alice,30,HR
Bob,24,IT
Charlie,28,Finance
```

Here's the code snippet that opens the output.csv file and defines a list containing all the data that is to be written into the CSV file.

Then, the csv.writer(file) object writes the data into the output.csv file by passing the list containing the header row and all subsequent data rows as arguments to the writerows() function:

```
import csv

# Data to be written to CSV
```

```
data = [  
    ["Name", "Age", "Department"],  
    ["Alice", 30, "HR"],  
    ["Bob", 24, "IT"],  
    ["Charlie", 28, "Finance"]  
]
```

```
# Open the CSV file in write mode  
with open('output.csv', mode='w', newline='') as file:  
    # Create a CSV writer object  
    csv_writer = csv.writer(file)  
  
    # Write the rows to the CSV file  
    csv_writer.writerows(data)
```

How to Manipulate and Analyze Data using CSV Files?

Data manipulation and analysis are important skills to have for any data analyst or scientist. In this section, we will take a look at basic data manipulation in Python.

The following are some data analysis things we can do with the CSV module in Python

Filtering Data

Filtering data involves selecting rows that meet certain criteria. For example, we might want to filter rows where the age of the employees is greater than 25.

Here's how we can filter the data for that:

```
filtered_data = [row for row in data if int(row['Age']) > 25]
```

```
# Display the filtered data  
for row in filtered_data:  
    print(row)
```

Adding a New Column

Adding a new column involves creating additional data based on the existing columns or with new data. For instance, we might want to add a column that calculates the age of each employee next year.

Here's how you can add a new column:

```
# Add a new column 'Age Next Year'  
for row in data:  
    row['age_next_year'] = int(row['Age']) + 1
```



```
# Display the data with the new column  
for row in data:  
    print(row)
```

This loop iterates over each row in the data list. In each row, it adds a new key-value pair where the key is 'age_next_year' and the value is the current age incremented by one. This creates a new column in the dataset.

We've successfully used the csv module to perform some basic operations on the files.

Conclusion

We explored CSV files and the csv module in this tutorial. Let's recap what we've discussed:

CSV files provide a standardized format for storing tabular data, ensuring compatibility across various platforms and applications.

Python's csv module offers efficient tools for reading, writing, and manipulating CSV files, simplifying data management tasks.

Data manipulation operations such as filtering rows and adding new columns can be easily performed using Python's csv module.

Python's simplicity and versatility make it a powerful tool for handling CSV files, facilitating seamless integration into data analysis workflows.

These were some basics of working with CSV files using Python. For more advanced data analysis tasks, libraries like Pandas can be integrated with Python to enhance CSV file processing capabilities.

CSV is a file format you will frequently come across while working in the field of [Data Science](#). It

is a type of text file that stores tabular data for better readability, easier understanding, and

faster processing. If you're wondering how to read CSV file in python, CSV files can be

converted from a JSON file or created using [Python](#) or Java.

In this article, we'll cover the basics of CSV files and how to handle them in Python. You'll learn

what a CSV file is in Python, and we'll show you how to read [CSV files](#) using [Python's](#) built-in

csv library. We'll also explain how to create a CSV file in Python and manage CSV file handling

easily. Whether you use the csv module or libraries like Pandas, this guide will help you read

CSV files in Python step-by-step. Plus, we'll touch on working with different data types and structures using Pandas and [NumPy](#) for efficient CSV handling.

Learning Outcomes

- Gain a fundamental understanding of what CSV files are and their importance in data science.
- Master how to read and write CSV files using Python's built-in csv library, including practical examples.
- Learn how to utilize the pandas library to read and write CSV files, enhancing your data manipulation skills with DataFrames and the pandas.read_csv function.
- Explore various methods to read CSV files in Python, such as using csv.reader, .readlines(), csv.DictReader, and pd.read_csv.
- Discover multiple ways to write data into CSV files using csv.writer, csv.DictWriter, .writelines(), and pandas, ensuring you can handle data export tasks effectively.
- Develop skills to parse data from CSV files, work with different data structures, and understand data types in the context of Python programming.
- Learn how to integrate libraries like NumPy with CSV file handling for more advanced data processing tasks in Python 3.

This article was published as a part of the [Data Science Blogathon](#).

Table of contents

1. What is a CSV?

- Structure of CSV in Python
- Read csv file in Python

2. How to Read CSV Files in Python with Procedural Steps?

- Steps to Read CSV Files in Python Using csv.reader

3. Implementing Code Using with() Statement

- How to Read CSV Files in Python Using .readlines()?
- How to Read CSV Files in python Using Pandas?
- Read CSV File in Python Using csv.DictReader

4. How to Write to a Python CSV?

- Write CSV file Using csv.writer
- Write CSV File Using .writelines()
- Write CSV Using Pandas
- Write CSV File Using csv.DictWriter

5. Conclusion

What is a CSV?

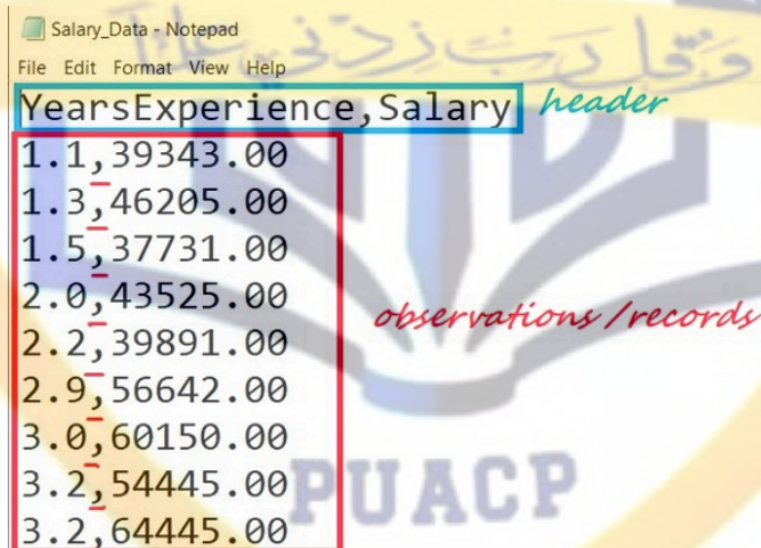
Understanding how to read CSV files in Python is essential for any data scientist. CSV, which stands for “Comma Separated Values,” serves as the fundamental format for storing tabular data as plain text. As data scientists, we frequently encounter CSV data in our daily workflows.

Therefore, mastering the ability to read CSV files in Python is crucial for efficiently handling and analyzing [data sets](#).

Structure of CSV in Python

We have a file named “Salary_Data.csv.” The first line of a CSV file is the header. It contains the names of the fields/features, which are shown on top as the column names in the file.

After the header, each line of the file is an observation/a record. The values of a record are separated by “commas.”



YearsExperience	Salary
1.1	39343.00
1.3	46205.00
1.5	37731.00
2.0	43525.00
2.2	39891.00
2.9	56642.00
3.0	60150.00
3.2	54445.00
3.2	64445.00

Read csv file in Python

There are two main ways to read CSV files in Python:

- Using the `csv` module: This is the built-in module for working with CSV files in Python. It provides basic functionality for reading and writing CSV data.

Here's an example of how to read a CSV file using csv.reader:

```
import csv

# Open the CSV file in read mode
with open('data.csv', 'r') as csvfile:
    # Create a reader object
    csv_reader = csv.reader(csvfile)

    # Iterate through the rows in the CSV file
    for row in csv_reader:
        # Access each element in the row
        print(row)
```

Copy Code

- Using the Pandas library: Pandas is a powerful library for data analysis in Python. It offers a more convenient way to read and manipulate CSV data.

Here's an example of how to read a CSV file using Pandas:

```
import pandas as pd

# Read the CSV file into a DataFrame
df = pd.read_csv('data.csv')

# Access data in the DataFrame using column names or indexing
print(df['column_name'])
print(df.iloc[0]) # Access first row
```

Copy Code

List of Methods to Read a CSV File in Python

- Read CSV file using csv.reader
- Read CSV file using .readlines() function
- Read CSV file using Pandas
- Read CSV file using csv.DictReader

How to Read CSV Files in Python with Procedural Steps?

There are many different ways to read data in a CSV file, which we will now see one by one.

Steps to Read CSV Files in Python Using csv.reader

You can read CSV files using the csv.reader object from Python's csv module. Steps to read a CSV file using csv reader:

1. Import the CSV library

```
import csv
```

2. Open the CSV file

The .open() method in python is used to open files and return a file object.

```
file = open('Salary_Data.csv')  
type(file)
```

The type of file is “_io.TextIOWrapper” which is a file object that is returned by the open() method.

3. Use the csv.reader object to read the CSV file

```
csvreader = csv.reader(file)
```

4. Extract the field names

Create an empty list called a header. Use the next() method to obtain the header.

The .next() method returns the current row and moves to the next row.

The first time you run `next()`, it returns the header, and the next time you run, it returns the first record, and so on.

```
header = []
```

```
header = next(csvreader)
```

```
header
```

```
['YearsExperience', 'Salary']
```

5. Extract the rows/records

Create an empty list called `rows` and iterate through the `csvreader` object and append each row to the `rows` list.

```
rows = []
```

```
for row in csvreader:
```

```
    rows.append(row)
```

```
rows
```

```
[['1.1', '39343.00'],  
 ['1.3', '46205.00'],  
 ['1.5', '37731.00'],  
 ['2.0', '43525.00'],  
 ['2.2', '39891.00'],  
 ['2.9', '56642.00'],  
 ['3.0', '60150.00'],  
 ['3.2', '54445.00'],  
 ['3.2', '64445.00'],  
 ['3.7', '57189.00'],  
 ['3.9', '63218.00'],  
 ['4.0', '55794.00'],  
 ['4.0', '56957.00'],  
 ['4.1', '57081.00'],  
 ['4.5', '61111.00'],  
 ['4.9', '67938.00'],  
 ['5.1', '66029.00'],  
 ['5.3', '83088.00'],  
 ['5.9', '81363.00'],  
 ['6.0', '93940.00'],  
 ['6.8', '91738.00'],  
 ['7.1', '98273.00'],  
 ['7.9', '101302.00'],  
 ['8.2', '113812.00'],  
 ['8.7', '109431.00'],  
 ['9.0', '105582.00'],  
 ['9.5', '116969.00'],  
 ['9.6', '112635.00'],  
 ['10.3', '122391.00'],  
 ['10.5', '121872.00']]
```

6. Close the file

`.close()` method is used to close the opened file. Once it is closed, we cannot perform any operations on it.

```
file.close()
```

Complete Code for Read CSV Python

```
import csv  
file = open("Salary_Data.csv")  
csvreader = csv.reader(file)  
header = next(csvreader)  
print(header)
```



```
rows = []
for row in csvreader:
    rows.append(row)
print(rows)
file.close()
Copy Code
```

Naturally, we might forget to close an open file. To avoid that, we can use the with() statement to automatically release the resources. In simple terms, there is no need to call the .close() method if we are using with() statement.

Implementing Code Using with() Statement

Basic Syntax: with open(filename, mode) as alias_filename:

Modes:

- 'r' – to read an existing file,
- 'w' – to create a new file if the given file doesn't exist and write to it,
- 'a' – to append to existing file content,
- '+' – to create a new file for reading and writing

```
import csv
rows = []
with open("Salary_Data.csv", 'r') as file:
    csvreader = csv.reader(file)
    header = next(csvreader)
    for row in csvreader:
        rows.append(row)
print(header)
print(rows)
Copy Code
```

```
[ 'YearsExperience', 'Salary' ]
[[ '1.1', '39343.00'], [ '1.3', '46205.00'], [ '1.5', '37731.00'], [ '2.0', '43525.00'], [ '2.2', '39091.00'], [ '2.9', '56642.00'], [ '3.0', '60150.00'], [ '3.2', '54445.00'], [ '3.2', '64445.00'], [ '3.7', '57189.00'], [ '3.9', '63218.00'], [ '4.0', '55794.00'], [ '4.0', '56957.00'], [ '4.1', '57081.00'], [ '4.5', '61111.00'], [ '4.9', '67938.00'], [ '5.1', '66029.00'], [ '5.3', '83088.00'], [ '5.9', '81363.00'], [ '6.0', '93940.00'], [ '6.8', '91738.00'], [ '7.1', '98273.00'], [ '7.9', '101302.00'], [ '8.2', '114812.00'], [ '8.7', '109431.00'], [ '9.0', '105582.00'], [ '9.5', '116969.00'], [ '9.6', '112635.00'], [ '10.3', '122391.00'], [ '10.5', '121872.00'] ]
```

Also Read: [The Evolution and Future of Data Science Innovation](#)

How to Read CSV Files in Python Using .readlines()?

Now the question is – “Is it possible to fetch the header and rows using only open() and with() statements and without the csv library?” Let’s see...

.readlines() method is the answer. It returns all the lines in a file as a list. Each item on the list is a row of our CSV file.

The first row of the file.readlines() is the header, and the rest are the records.

with open('Salary_Data.csv') as file:

```
    content = file.readlines()
```

```
header = content[:1]
```

```
rows = content[1:]
```

```
print(header)
```

```
print(rows)
```

Copy Code

```
[ 'YearsExperience,Salary\n' ]
[ '1.1,39343.00\n', '1.3,46205.00\n', '1.5,37731.00\n', '2.0,43525.00\n', '2.2,39091.00\n', '2.9,56642.00\n', '3.0,60150.00\n', '3.2,54445.00\n', '3.2,64445.00\n', '3.7,57189.00\n', '3.9,63218.00\n', '4.0,55794.00\n', '4.0,56957.00\n', '4.1,57081.00\n', '4.5,61111.00\n', '4.9,67938.00\n', '5.1,66029.00\n', '5.3,83088.00\n', '5.9,81363.00\n', '6.0,93940.00\n', '6.8,91738.00\n', '7.1,98273.00\n', '7.9,101302.00\n', '8.2,114812.00\n', '8.7,109431.00\n', '9.0,105582.00\n', '9.5,116969.00\n', '9.6,112635.00\n', '10.3,122391.00\n', '10.5,121872.00\n' ]
```

**The ‘n’ from the output can be removed using .strip() method.

What if we have a huge dataset with hundreds of features and thousands of records? Would it be possible to handle lists??

Here comes pandas library into the picture.

How to Read CSV Files in python Using Pandas?

Let's have a look at how pandas are used to read data in a CSV file.

Step1: Import pandas library

```
import pandas as pd
```

Copy Code

Step2: Load CSV files to pandas using read_csv()

Basic Syntax: `pandas.read_csv(filename, delimiter=',')`

```
data= pd.read_csv("Salary_Data.csv")
```

data

Copy Code



	YearsExperience	Salary
0	1.1	39343.0
1	1.3	46205.0
2	1.5	37731.0
3	2.0	43525.0
4	2.2	39891.0
5	2.9	56442.0
6	3.0	60150.0
7	3.2	54445.0
8	3.2	64445.0
9	3.7	57169.0
10	3.9	63218.0
11	4.0	55794.0
12	4.0	56957.0
13	4.1	57081.0
14	4.5	61111.0
15	4.9	67938.0
16	5.1	66029.0
17	5.3	83088.0

Step3: Extract the field names

.columns is used to obtain the header/field names.

```
data.columns
```

Copy Code

```
Index(['YearsExperience', 'Salary'], dtype='object')
```

Step4: Extract the rows

All the data of a data frame can be accessed using the field names.

```
data.Salary
```

[Copy Code](#)



0	39343.0
1	46205.0
2	37731.0
3	43525.0
4	39891.0
5	56642.0
6	60150.0
7	54445.0
8	64445.0
9	57189.0
10	63218.0
11	55794.0
12	56957.0
13	57081.0
14	61111.0
15	67938.0
16	66029.0
17	83088.0
18	81363.0
19	93940.0
20	91738.0
21	98273.0
22	101302.0
23	113812.0
24	109431.0
25	105582.0
26	116969.0
27	112635.0
28	122391.0
29	171872.0

Name: Salary, dtype: float64

Read CSV File in Python Using csv.DictReader

A dictionary in how to Read CSV file in Python is like a hash table, containing keys and values.

To create a dictionary, you use the dict() method with specified keys and values. If you're working with CSV files in Python, the csv module's .DictReader comes in handy for reading them. Here's a simple guide on how to use Python to read CSV file

Step1: Import the csv module

```
import csv
```

[Copy Code](#)

Step2: Open the CSV file using the .open() function with the mode set to 'r' for reading.


```
with open('Salary_Data.csv', 'r') as csvfile:
```

Copy Code

Step3: Create a DictReader object using the csv.DictReader() method.

```
reader = csv.DictReader(csvfile)
```

Copy Code

Step4: Use the csv.DictReader object to read the CSV file.

Iterate through the rows of the CSV file using a 'for' loop and the DictReader object to see the field names as keys along with their respective values.

```
for row in reader:
```

```
    print(row)
```

Copy Code

List of Methods to Write a CSV file in python

- Write CSV file using csv.writer
- Write CSV file using writelines() function
- Write CSV file using Pandas
- Write CSV file using csv.DictWriter

How to Write to a Python CSV?

We can write to a CSV file in multiple ways.

Write CSV file Using csv.writer

The csv.writer() function returns a writer object that converts the input data into a delimited string.

For example, let's assume we are recording the data of 3 students (Name, M1 Score, M2 Score)

```
header = ['Name', 'M1 Score', 'M2 Score']
data = [['Alex', 62, 80], ['Brad', 45, 56], ['Joey', 85, 98]]
```

[Copy Code](#)

Now let's see how this data can be written to a CSV file using csv.writer:

Step1: Import csv library.

```
import csv
```

[Copy Code](#)

Step2: Define a filename and Open the file using open().

Step3: Create a csvwriter object using csv.writer().

Step4: Write the header.

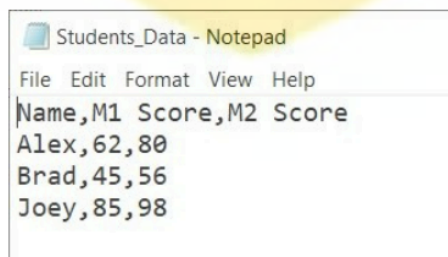
Step5: Write the rest of the data.

Code for steps 2-5

```
filename = 'Students_Data.csv'
with open(filename, 'w', newline='') as file:
    csvwriter = csv.writer(file) # 2. create a csvwriter object
    csvwriter.writerow(header) # 4. write the header
    csvwriter.writerows(data) # 5. write the rest of the data
```

[Copy Code](#)

Below is how our CSV file looks.



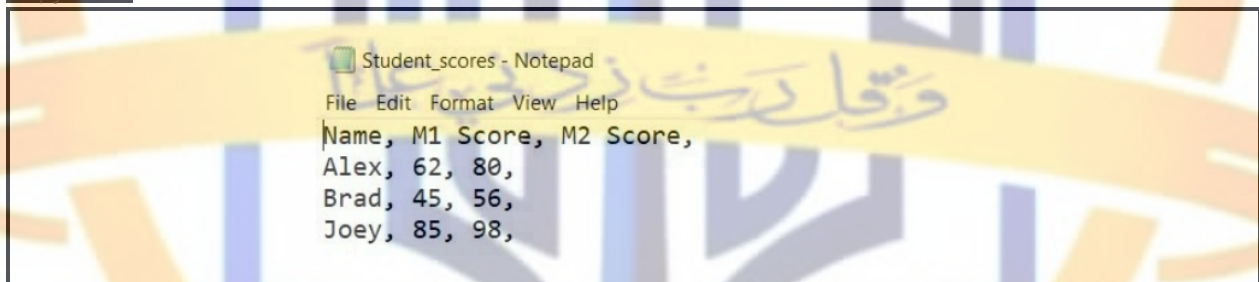
```
Students_Data - Notepad
File Edit Format View Help
Name,M1 Score,M2 Score
Alex,62,80
Brad,45,56
Joey,85,98
```

Write CSV File Using .writelines()

.writelines() iterates through each list, converts the list elements to a string, and then writes it to the csv file.

```
header = ['Name', 'M1 Score', 'M2 Score']
data = [['Alex', 62, 80], ['Brad', 45, 56], ['Joey', 85, 98]]
filename = 'Student_scores.csv'
with open(filename, 'w') as file:
    for header in header:
        file.write(str(header)+'\n')
    for row in data:
        for x in row:
            file.write(str(x)+'\n')
            file.write('\n')
```

Copy Code



Write CSV Using Pandas

Follow these steps to write to a CSV file using pandas:

Step1: Import pandas library

```
import pandas as pd
```

Copy Code

Step2: Create a pandas dataframe using pd.DataFrame

Syntax: pd.DataFrame(data, columns)

The data parameter takes the records/observations, and the columns parameter takes the columns/field names.

```
header = ['Name', 'M1 Score', 'M2 Score']
data = [['Alex', 62, 80], ['Brad', 45, 56], ['Joey', 85, 98]]
data = pd.DataFrame(data, columns=header)
Copy Code
```

Step3: Write to a CSV file using to_csv()

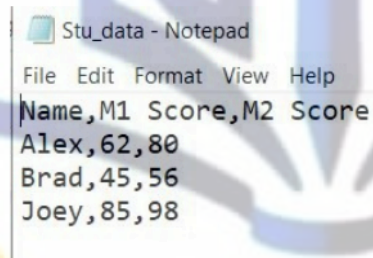
Syntax: DataFrame.to_csv(filename, sep=',', index=False)

**separator is ',' by default.

index=False to remove the index numbers.

```
data.to_csv('Stu_data.csv', index=False)
Copy Code
```

Below is how our CSV looks like



```
Stu_data - Notepad
File Edit Format View Help
Name,M1 Score,M2 Score
Alex,62,80
Brad,45,56
Joey,85,98
```

Write CSV File Using csv.DictWriter

You can write data into a CSV file using the csv module .DictReader following the below steps.

Step1: Import the csv module

```
import csv
Copy Code
```

Step2: Using the .open() function, create a new file object with the mode as 'w' for writing

Create a new file object using the `open()` function, specifying the file name with the mode set as 'w' for writing.

```
with open('Students_Data.csv', 'w', newline='') as csvfile:
```

Copy Code

Step3: Type in the data you want to write to the CSV file as a list of dictionaries

```
data = [{'Name': 'Alex', 'M1 Score': 62, 'M2 Score': 80},  
        {'Name': 'Brad', 'M1 Score': 45, 'M2 Score': 56},  
        {'Name': 'Joey', 'M1 Score': 85, 'M2 Score': 98}]
```

Copy Code

Step4: Create a `csv.DictWriter` object specifying the file object, the fieldname parameters, and the delimiter

Note that the delimiter by default is ','

```
fieldnames = ['Name', 'M1 Score', 'M2 Score'] writer = csv.DictWriter(csvfile, fieldnames=fieldnames)
```

Copy Code

Step5: Write the header row using the `writeheader()` method.

```
writer.writeheader()
```

Copy Code

Step6: Use the `writerows()` method to write the data to the CSV file

```
writer.writerows(data)
```

Copy Code

This will create a new file named 'Students_Data.csv' with Name, M1 Score, and M2 Score as the header/column names and the data values under the data variable.

Conclusion

By now, I'm sure you are all familiar with the various techniques for handling CSV files in

Python, including the essential process of [Python](#) read CSV file. We trust this article has been

informative for all. Feel free to share it with your study buddies to spread the knowledge and enhance everyone's Python skills.

Knowing how to read and write [CSV files](#) in Python is an essential skill for any data scientist or analyst. It can save time, improve productivity, and make data processing more efficient.

Whether you're just starting out or looking to take your skills to the next level, our [Data Science Black Belt program](#) is an excellent resource to enhance your knowledge in data science. The program covers basics of Python programming to advanced machine learning concepts. With hands-on projects and case studies, you'll gain practical experience and learn how to apply your skills to real-world problems.

Key Takeaways

- Creating a Comma Separated Values (CSV) file is the simplest way of converting complex data into a readable text file.
- A file in the CSV format shows you organized tabular data similar to an excel sheet.
- You can read a CSV file in Python using `csv.reader`, `.readlines()`, or `csv.DictReader`, and write into one by using `.writer`, `.DictWriter`, or `.writelines()`.
- Pandas can be used for both reading and writing data in a CSV.

<https://www.analyticsvidhya.com/blog/2021/08/python-tutorial-working-with-csv-file-for-data-science/>